

3_Objektově orientované programování (OOP)

Definice: Programovací paradigma kde kód skládáme z objektů. **Třída** je šablona která definuje atributy a metody. **Objekt** je instance třídy — uchovává konkrétní hodnoty atributů, metody sdílí ze třídy.

Základní pojmy

- **Třída** — šablona/vzor pro objekty. Existuje v kódu, nezabírá paměť za běhu.
- **Objekt** — konkrétní instance třídy. Vzniká za běhu, zabírá paměť.
- **Atribut** — vlastnost objektu (data která objekt uchovává)
- **Metoda** — funkce definovaná uvnitř třídy, pracuje s atributy objektu
- **Konstruktor** — speciální metoda volaná při vytváření objektu, nastavuje počáteční hodnoty atributů

csharp

```
class Auto
{
    public string Barva { get; set; }
    public string Znacka { get; set; }

    public Auto(string barva, string znacka, int rok = 2024) // konstruktor
    {
        Barva = barva;
        Znacka = znacka;
    }
}

Auto a = new Auto("červená", "Škoda"); // objekt = instance třídy
```

Statické vs. Instanční atributy

- **Instanční** — patří konkrétnímu objektu, každý objekt má svou hodnotu
- **Statické** (`static`) — patří třídě, sdílené všemi objekty

csharp

```
class Auto
{
    public string Barva { get; set; }    // instanční - každé auto má jinou
    public static int PocetKol = 4;      // statický - stejné pro všechna
    auta
}

Console.WriteLine(Auto.PocetKol); // přístup přes třídu, ne objekt!
```

4 pilíře OOP

1. Zapouzdření

Data objektu jsou ideálně skrytá (`private` / `protected`), přistupuje se k nim pouze přes gettery a settery. Umožňuje validaci dat při zápisu.

- `private` — přístupné pouze ve třídě
- `protected` — přístupné ve třídě a jejích dětech
- `public` — přístupné odkudkoliv

csharp

```
class Osoba
{
    private int _vek;

    public int Vek
    {
        get { return _vek; }
        set
        {
            if (value < 0) throw new Exception("Věk nemůže být záporný");
            _vek = value;
        }
    }
}
```

2. Dědičnost

Třída (dítě) přebírá atributy a metody jiné třídy (rodiče). V C# lze dědit pouze od **jedné** třídy. Dítě může metody rodiče přepsat pomocí `override`. Rodič musí metodu označit jako `virtual`.

csharp

```
class Zvire
{
    public string Jmeno { get; set; }
    public virtual void Zvuk() { Console.WriteLine("..."); } // virtual =
    může být přepsáno
}

class Pes : Zvire
{
    public override void Zvuk() { Console.WriteLine("Haf!"); }
}
```

3. Polymorfismus

Statický (overloading) — stejný název metody, různé parametry → řeší se při **kompilaci**

csharp

```
void Secti(int a, int b) { }
void Secti(string a, string b) { }
```

Dynamický (override) — děti přepisují metodu rodiče, přistupujeme k nim přes rodičovský typ → řeší se za **běhu**

csharp

```
List<Tvar> tvary = new List<Tvar> { new Kruh(), new Ctverec() };
foreach (Tvar t in tvary) { t.Obsah(); } // za běhu se rozhodne která metoda
```

4. Abstrakce

Třída definuje **co** objekt umí, ne jak to dělá. Implementaci si řeší každá podtřída sama. Realizuje se pomocí **abstraktních tříd** nebo **rozhraní (interface)**.

Abstraktní třídu **nelze instanciovat** — nelze z ní přímo vytvořit objekt.

csharp

```
abstract class Tvar
{
    public abstract double Obsah(); // předpis - každý tvar si řeší sám
}

class Kruh : Tvar
{
    public double Polomer { get; set; }
    public override double Obsah() { return Math.PI * Polomer * Polomer; }
}
```

Abstraktní třída vs. Interface

	Abstraktní třída	Interface
Hotový kód	✓	✗
Atributy	✓	✗
Dědičnost	pouze jedna	více najednou
Použití	třídy sdílí kód	třídy sdílí chování

csharp

```
interface ILetajici { void Letej(); }
interface IZpivajici { void Zpivej(); }

class Ptak : Zvire, ILetajici, IZpivajici { } // jedna třída + více
interfaců
```

SOLID principy

Písmeno	Princip	Jednoduše
S	Single Responsibility	Třída má jen jednu zodpovědnost

Písmeno	Princip	Jednoduše
O	Open/Closed	Rozšiřuj, neupravuj
L	Liskov Substitution	Dítě funguje všude kde funguje rodič
I	Interface Segregation	Více malých interfaců > jeden velký
D	Dependency Inversion	Záviset na abstrakci, ne na konkrétní třídě